

0.2 - MACHINE LEARNING REVISION

CHAPTER INDEX

- ☐ [CORE MACHINE LEARNING CONCEPTS](#)
 - [CLASSIFICATION](#)
 - [REGRESSION](#)
 - [CLUSTERING](#)
 - [DIMENSIONALITY REDUCTION](#)
 - [FEATURE LEARNING](#)
 - [LEARNING APPROACHES](#)
 - [SPLIT COMPUTING](#)
 - [ONE/ZERO SHOT LEARNING](#)
 - [SEQUENCE LEARNING](#)
 - [REINFORCEMENT LEARNING](#)
 - [PARAMETERS OPTIMIZATION](#)
 - [HYPERPARAMETERS](#)
- ☐ [PERFORMANCE OF ML MODELS](#)
 - [OVERFITTING, UNDERFITTING AND GROKING](#)
 - [OPEN SET PERFORMANCE INDICATORS](#)
 - [TRAINING, VALIDATION AND TESTING](#)
 - [CONVERGENCE](#)

CORE MACHINE LEARNING CONCEPTS

Though machine learning is *not a strictly mandatory prerequisite* for deep learning, a foundational knowledge of its concepts is *highly recommended* and widely considered best practice.

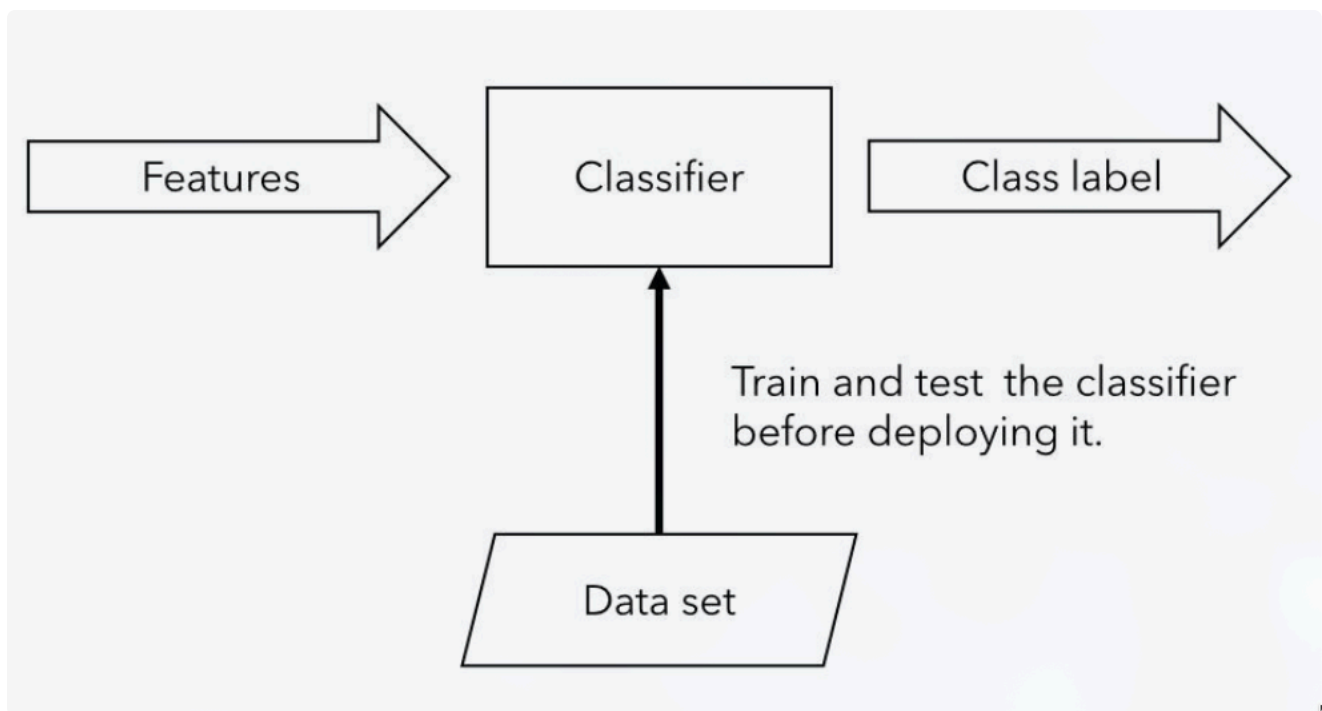
Thus, this section aims at building a *baseline understanding* of the *most relevant ML topics*, which will greatly boost the reader's ability to comprehend the topics that will be presented in the course.

CLASSIFICATION

Classification is the problem of *assigning a class to a pattern*, with the use of a **classifier**.

CLASSIFIER

Any *function, method or algorithm* that assigns a **class label** to a *pattern*.



🔍 OBSERVATION

The "class" concept is *strictly dependent* on the *application domain*:

- 21 classes for the *recognition of letters* of the latin alphabet (one for each letter).
- 2 classes to *distinguish latin letters from cyrillic letters* (a letter is from either one or the other).

✍️ NOTE

If there are only **two classes**, the term **binary classification** is used, otherwise, in presence of multiple classes, the problem is referred to as **multi-class classification**.

Some examples of classification problems are:

- Spam detection.
- Credit card fraud detection.
- Face recognition.
- Pedestrian classification.
- Medical diagnosis.
- Stock trading.

✍️ MULTILABEL CLASSIFICATION

In some applications, *several classes are associated with each pattern*: these problems are referred to with the term **multilabel classification**.

An example of such problem is classifying a newspaper article based on the *different topics it covers*.

REGRESSION

Regression *assigns a continuous* value to a pattern*, with the goal of *predicting quantities*.

REGRESSION PROBLEM

A regression problem deals with the **prediction of a quantity**.

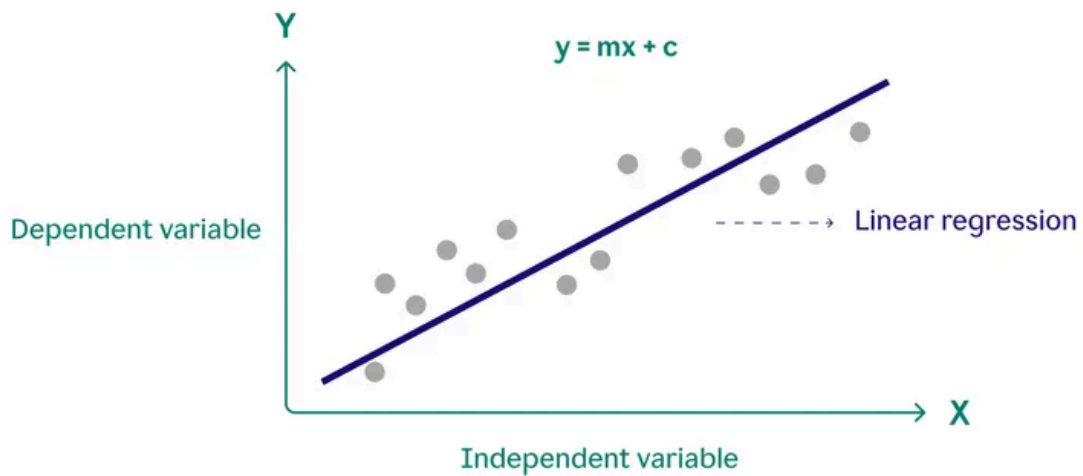
If *multiple input variables* are present, we use the term **multivariate regression**; furthermore, if input variables are ordered in time, we use the term **time series forecasting**.

NOTE*

Input variables can take *continuous* or *discrete values*, but in both cases the *output prediction* will be continuous.

Given a data set of pairs of the form (input variable, value to predict), *solving* a regression problem corresponds to **learning a function** that **approximates the pairs**. Once this function is learned, we can use it to *predict the values of new input variables*.

A simple example of *regression* is given by **linear regression**, which uses a *linear function* to approximate the data set.



💡 REGRESSION IN GENERAL

Note that if the points in the data set are *scattered in more complicated ways*, linear regression may not be a good way to approximate the data. Instead, *other types of regression* will be employed (such as polynomials of higher degree).

Example applications of regression are:

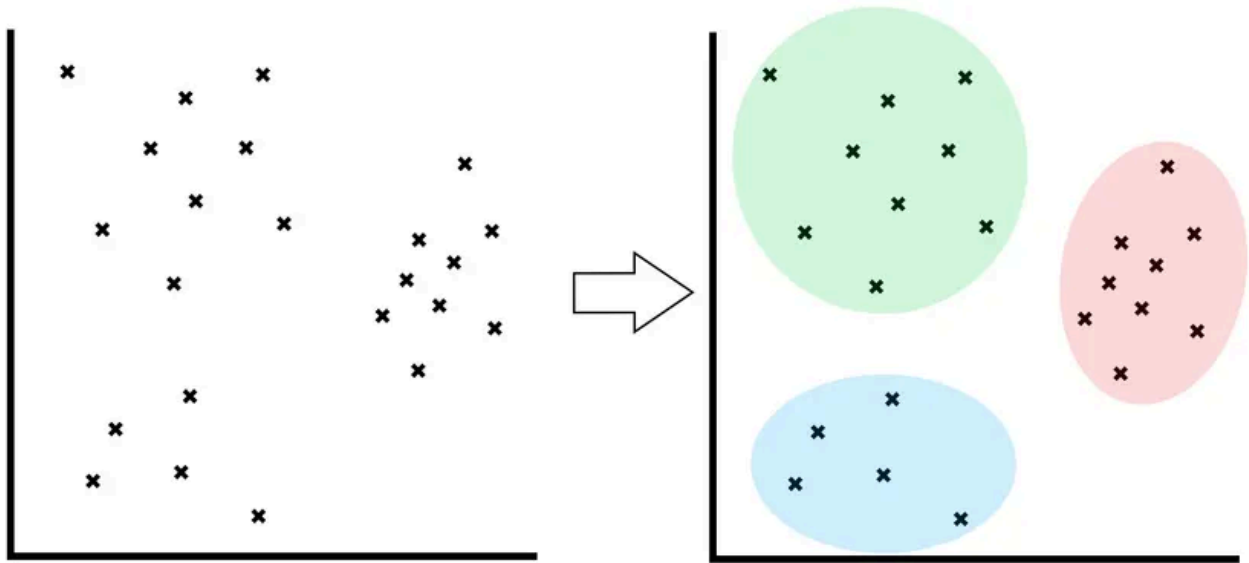
- Real estate prices prediction.
- Risk estimation for insurance companies.
- Energy consumption prediction to optimise the distribution grid.

CLUSTERING

📖 CLUSTERING PROBLEM

The goal is to **identify groups** (*clusters*) of **patterns with similar characteristics**.

The difficulty lies in the fact that **classes are not known** and **labels are not assigned**: the *unsupervised nature* of the problem makes it inherently more complex.



NOTE

1. Often, not even the *number of clusters* is known beforehand.
2. The identified clusters can subsequently be *used as classes*.

Example applications of clustering come up in:

- Marketing: definition of user groups based on consumption habits.
- Genetics: grouping of individuals on the basis of DNA analogies.
- Computer vision: image segmentation.

DIMENSIONALITY REDUCTION

Dimensionality reduction techniques are often employed to solve the so-called **curse of dimensionality**, which refers to various phenomena that arise when analysing data in *high-dimensional spaces* (often with hundreds or thousands of dimensions) that otherwise do not occur in low-dimensional settings.

CURSE OF DIMENSIONALITY (MAIN PROBLEM)

When the **dimensionality** of the data **increases**, the volume of the space increases so fast that the available **data becomes sparse**.

This sparsity is problematic for any method that requires *statistical significance*.

In order to obtain a statistically sound and reliable result, the *amount of data needed* to support the result often *grows exponentially* with the dimensionality.

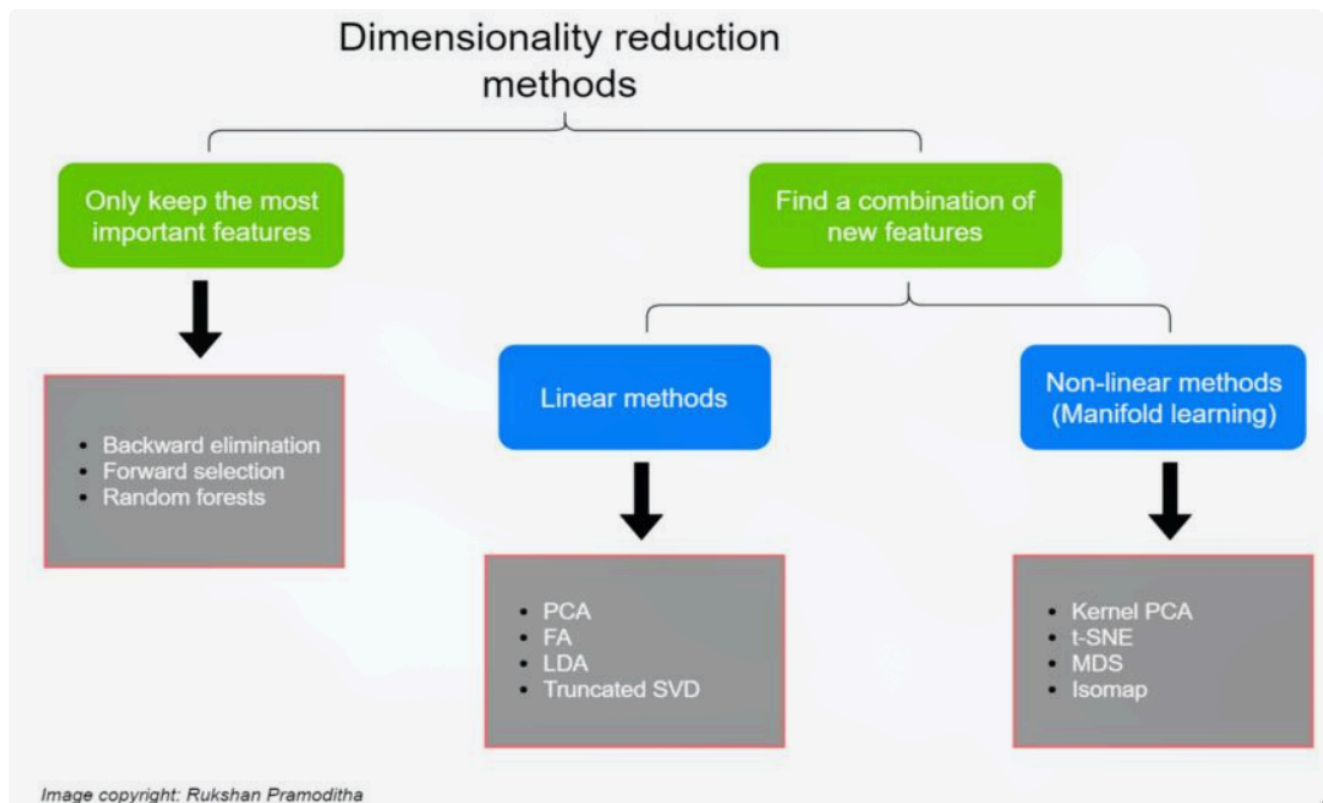
Also, *organising and searching data* often relies on detecting areas where objects form *groups with similar properties*; in high dimensional data, however,

all *objects appear to be sparse*, which prevents common data organisation strategies from being efficient.

In practice, dimensionality reductions are **mappings** φ :

$$\varphi : \mathbb{R}^d \rightarrow \mathbb{R}^k, k < d$$

In simple terms, they *only use* k features to describe a pattern instead of the original d features: this is often made possible by the fact that *not all the original features are useful/relevant*.



FEATURE LEARNING

The success of many machine learning applications depends from the **effectiveness of the representation of the patterns** in terms of features.

FEATURE ENGINEERING

The **definition of ad-hoc** (hand-crafted) **features** to **ease learning approaches** in many different application domains.

For example, for the recognition of objects, numerous *shape, colour and texture descriptors* have been crafted and we can use them to **convert images to numeric vectors**.

See > [0.1 - PATTERN REPRESENTATION > LOCAL BINARY PATTERNS](#) for an example of such descriptors.

An interesting question that arises is the following:

💡 REPRESENTATION LEARNING

Can we **automatically learn effective features** from raw data?

That is, can we **operate directly on raw data** (e.g. RGB intensity of each pixel in an image), **without using pre-defined features**?

Deep learning techniques operate exactly this way: they use raw data as input and *automatically extract features* necessary to solve the problem.

LEARNING APPROACHES

Different learning approaches are used depending on the *nature of the training data*:

📖 LEARNING APPROACHES

- **SUPERVISED LEARNING**: pattern *classes are known* and the training set is *labelled*.
- **UNSUPERVISED LEARNING**: the *classes of the patterns* in the training set are *unknown* and the training data is *unlabelled*.
- **SEMISUPERVISED LEARNING**: the training set is *only partially labelled*.
- **UNLEARNING**: a technique to *selectively remove* the influence of specific training data points on the model.

More formally, in **supervised learning** we have *pre-classified examples* (training set):

$$\text{Tr} = \{(x^{(i)}, t^{(i)})\}$$

and we want to *learn a general description* that captures the information content of the examples.

Assuming that the *true labels* are given by:

$$t^{(i)} = f(x^{(i)}) + \epsilon^{(i)}$$

the model will *learn a function* h such that:

$$h(x^{(i)}) \approx f(x^{(i)}), \forall i \in \text{Tr}$$

It is then possible to use this description in a *predictive way*; that is, given a new input $\hat{x}$, predict the associated output $h(\hat{x})$ (Hopefully $h(\hat{x}) \approx \hat{x}^*$, where $\hat{x}^*$ is the *true label*).

In **unsupervised learning**, the training set is *unlabelled*:

$$\text{Tr} = \{x^{(i)}\}$$

And we want to *discover regularities and patterns*, without the help of a supervisor that provides labels.

In both cases, the model learns a function h that we call **hypothesis**.

Consider, for example, the *classification* of points $p \in \mathbb{R}^2$ into *two classes* (binary classification). If the two regions are *roughly separated by a straight line* of the form:

$$\vec{w} \cdot \vec{x} + b = 0, \vec{w} \in \mathbb{R}^2, b \in \mathbb{R}$$

then the *learned hypothesis* will be:

$$\mathcal{H}(\vec{x}) = \text{sign}(\vec{w} \cdot \vec{x} + b)$$

That is, points will be given labels $l \in \{-1, +1\}$ depending on *which side of the line they are located*.

Those presented above are just the "*main*" approaches to learning.

Clearly, other more specific forms of learning exist; some of which are introduced below:

OTHER LEARNING APPROACHES

- **ACTIVE LEARNING:** particular case of semisupervised learning. At each iteration, some *unlabelled patterns* are chosen and *labelled*; these are often the most difficult to be classified, which are unlikely to be "learnt" by the model alone without any form of supervision.
- **PSEUDO LEARNING:** labels are assigned by a *classifier* instead of having humans label them.
- **SELF - SUPERVISED LEARNING:** a form of unsupervised learning where the *supervision is provided by the data* itself. That is, models are trained in such a way that *output vectors* of related data in the training set are "*close to each other*", while output vectors of *unrelated data points* are "*distant*", but without ever assigning labels.

Furthermore, learning approaches can also be classified based on *how the training set is used*:

LEARNING AND TRAINING SET

- **BATCH TRAINING:** the training is done *only once* on the training set. At the end of the training process, the model is "*frozen*" and *stops learning*.
- **INCREMENTAL TRAINING:** the training is *split in different sessions*. This approach introduces the risk of *catastrophic forgetting* (the system *forgets* what it had learned earlier).
- **FEDERATED LEARNING:** the model is trained on *multiple local datasets* contained in *different nodes* without explicitly exchanging data samples. The approach is used when *privacy is a concern*; thus, parameters are

exchanged between nodes at some frequency to generate a "*global model*".

- **DISTRIBUTED LEARNING:** a single model is *trained on multiple servers*. The key distinction with federated learning is that distributed learning assumes that the *datasets are independent and identically distributed* (in federated learning there are no assumptions about the distributions of the local datasets).

SPLIT COMPUTING

The Deep Neural Network (DNN) *computational requirements* **preclude their device-only deployment** on most of the *resource-constrained systems*, such as **mobile phones**.

A possible alternative is to refer to **simplified models**, with an obvious **accuracy decline** which may not be desirable.

The opposite **cloud-only paradigm** is to *transfer the data captured* by the device to a *server*

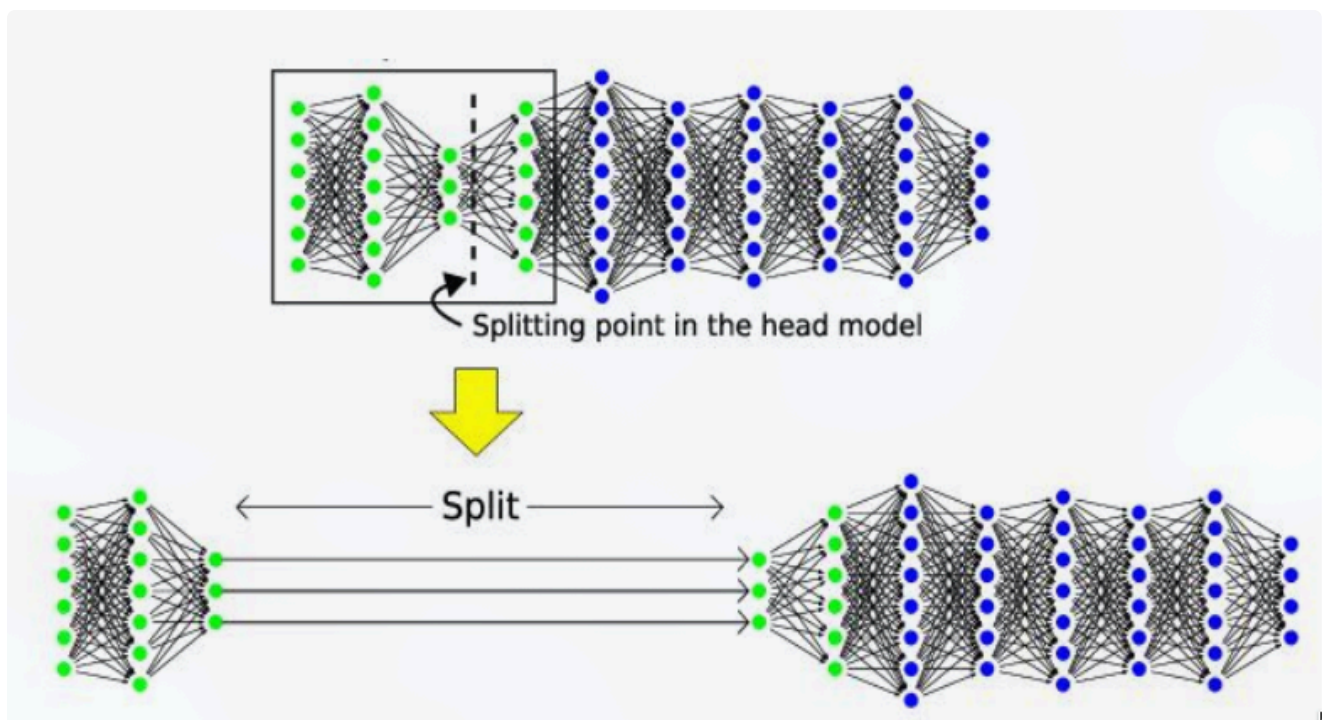
through a communication network. The servers, which run the DNN model, compute the inference and *send it back to the device*.

In this case, the **high data transfer time** and possible **server congestion** are obvious downsides.

As a compromise between the device-only and the cloud-only approaches, the **split computing (SC)** frameworks propose to **divide DNN models into a "head" and a "tail"**, deployed at the edge device and the server, respectively.

Using SC, the *computational load is distributed* across the two platforms, exploiting the *computational power of both devices* at the cost of transferring data on a network connection.

Data transfer effort is mitigated by the injection of bottlenecks, usually carried out by *autoencoders*, achieving in-model compression.



Setting up a SC system requires **setting up the optimal splitting point**: so far, this choice was driven primarily by *architectural considerations* such as *memory capabilities* of the edge device, *desired transmission rate* between head and tail, *compression rate* of the bottleneck, and nature of the layers.

ONE/ZERO SHOT LEARNING

We've seen how models are trained on datasets. These datasets are usually huge, but some applications require the model to "*learn on the fly*"; this problem is known as *one shot learning*.

📖 ONE/FEW SHOT LEARNING

One/few-shot learning is an **object categorization problem**, found mostly in computer vision that aims to **classify objects from one, or only a few, samples**.

The ability to learn object categories from few examples, and at a rapid pace, has been *demonstrated in humans*.

It is estimated that a child learns almost all of the 10 ~ 30 thousand object categories in the world by age six. This is due *not only to the human mind's computational power*, but also to **its ability to synthesise and learn new object categories from existing information** about different, previously learned categories.

Given two examples from two object categories: one, an unknown object composed of familiar shapes, the second, an unknown, amorphous shape; it is much easier for humans to recognise the former than the latter, suggesting that humans **make use of previously learned categories** when learning new ones.

The key motivation for solving one-shot learning is that systems, like humans, can **use knowledge about object categories to classify new objects**.

A similar problem is **zero shot learning**.

To understand it, consider the following example: suppose *a person knows what a horse looks like*, and give that person the info that "a zebra is a horse with black and white striped skin". Then there is a **high chance** that the person can **identify** a zebra even though that person **had no prior idea** of what a zebra is.

Zero-shot learning is an approach in machine learning that takes inspiration from this.

📖 ZERO SHOT LEARNING

Learning approach in which data is arranged as follows:

- **SEEN CLASSES:** classes with labels available for training.
- **UNSEEN CLASSES:** classes that only *occur during testing or inference*, but *not during training*.
- **AUXILIARY INFORMATION:** information about both seen and unseen classes, that can help the model recognise unseen classes the first time it encounters them.

SEQUENCE LEARNING

So far, we have looked at problems where we have some **fixed number of inputs** and produce a **fixed number of outputs**. For example, we considered predicting house prices from a fixed set of features: square footage, number of bedrooms, number of bathrooms, walking time to downtown.

We also discussed mapping from an image (of fixed dimension) to the predicted probabilities that it belongs to each of a fixed number of classes, or taking a user ID and a product ID, and predicting a star rating.

In these cases, once we feed our fixed-length input into the model to generate an output, **the model immediately forgets what it just saw**.

This might be fine if our inputs truly all have the same dimensions and if successive inputs truly have nothing to do with each other.

But **how would we deal with video snippets?**

In this case, each snippet might consist of a **different number of frames** and our **guess** of what is going on in each frame might be **much stronger if we take into account the previous or succeeding frames**.

The same goes for language. One popular deep learning problem is machine translation: the task of *ingesting sentences in some source language* and *predicting their translation in another language*.

These problems also occur in medicine.

We might want a model to **monitor patients in the intensive care unit** and to fire off alerts if their risk of death in the next 24 hours exceeds some threshold. We definitely would **not want this model to throw away everything it knows** about the patient

history each hour and just make its predictions based on the most recent measurements.

These problems are among the most exciting applications of machine learning and they are instances of **sequence learning**.

They require a model to either **ingest sequences of inputs** or to **emit sequences of outputs** (or both). Specifically, **sequence to sequence learning** considers problems where **input and output are both variable-length sequences**, such as machine translation and transcribing text from the spoken speech.

REINFORCEMENT LEARNING

Considering the **interaction with an environment** opens a whole set of new modelling questions.

The following are just a few examples.

- Does the **environment remember** what we did previously?
- Does the **environment want to help us**, e.g., a user reading text into a speech recogniser?
- Does the **environment want to beat us**, i.e., an adversarial setting like spam filtering (against spammers) or playing a game (vs. an opponent)?
- Does the environment **not care**?
- Does the environment have **shifting dynamics**? For example, does future data always resemble the past or do the *patterns change over time*, either naturally or in response to our automated tools?

This last question raises the problem of **distribution shift**, when training and test data are different.

If you are interested in using machine learning to develop an agent that interacts with an environment and takes actions, then you are probably going to wind up focusing on **reinforcement learning**.

This might include applications to *robotics*, to dialogue systems, and even to developing artificial intelligence (AI) for video games.

Deep reinforcement learning, which *applies deep learning to reinforcement learning* problems, has surged in popularity. The breakthrough *deep Q-network* that beat humans at Atari games using only the visual input, and the AlphaGo program that dethroned the world champion at the board game Go are two prominent examples.

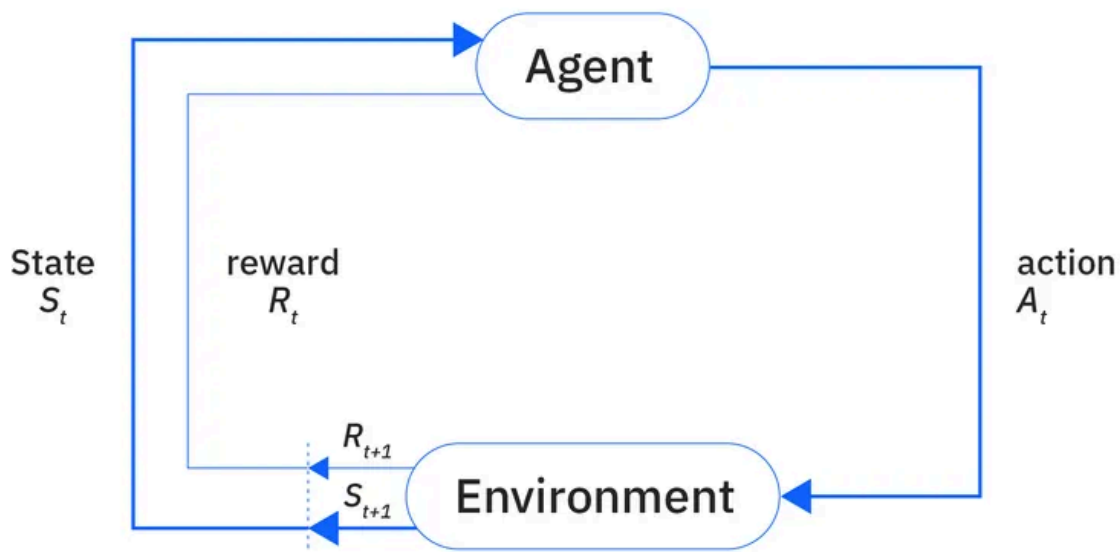
Reinforcement learning gives a very **general statement of a problem**, in which an **agent interacts with an environment** over a series of time steps.

At each time step, the agent receives some *observation* from the environment and must **choose an action** that is subsequently transmitted back to the environment via some mechanism (sometimes called an *actuator*).

Finally, the agent receives a **reward from the environment**.

The agent then receives a subsequent observation, and chooses a subsequent action, and so on.

The **behaviour** of a reinforcement learning agent is **governed by a policy**. In short, a policy is just a function that *maps from observations of the environment to actions*. The **goal** of reinforcement learning is to **produce a good policy**.



🔍 FIGURE EXPLANATION

At each time step t , the agent **performs an action** A_t that has an *effect on the environment*, currently in state S_t . In turn, the environment *responds* with a **new state** S_{t+1} and a **reward** R_{t+1} that the agent uses to *choose the next action*.

✍ NOTE

In practical, real world applications, it is *very difficult to obtain examples* that are both correct and representative of *all situations* in which the agent must work.

Therefore the classic **supervised approach** is not usually applicable.

PARAMETERS OPTIMIZATION

Typically, the behaviour of a Machine Learning algorithm is **regulated by a set of parameters** (e.g. the **weights** of the connections in a neural network).

Learning consists in determining the **optimal value of these parameters**.

Given a *training set* Tr and a *set of parameters* Θ , the **objective** can indicate the **optimality of the solution**, which is to be maximised:

$$\Theta^* = \operatorname{argmax}_{\Theta} f(Tr, \Theta)$$

Though in many cases, the parameters are chosen to **minimise the error** (referred to as *loss function*):

$$\Theta^* = \operatorname{argmin}_{\Theta} f(\operatorname{Tr}, \Theta)$$

In both cases, the function f can be optimised:

- **Explicitly**, with methods that operate on the very mathematical definition of f itself (E.g. calculating its partial derivatives and finding minima and maxima).
- **Implicitly**, with heuristic approaches. These are algorithms designed to find *approximate solutions* when *exact solutions can't be computed*, either because the problem doesn't have a known analytical solution, or because computing it would be too slow.

HYPERPARAMETERS

Many algorithms require to define, before beginning the learning phase, the value of the so-called **hyperparameters**.

Examples of hyperparameters are:

- Total *number of neurons in a network*.
- The *number of neighbours* in a k -NN classifier.
- The *degree of a polynomial* used in a *regression* approach.
- The *type of loss function*.

Usually, one would proceed with a *two-levels approach*: for each "reasonable" value of the hyperparameters, a learning phase is performed, producing *different versions of the model*. At the end, a *validation test* is conducted and the **hyperparameters are chosen from the better performing model**.

Another possible approach is **grid search**: for each hyperparameter we *define a set of values to try* and the system is then **evaluated on all combinations of these values**. Clearly, this approach can quickly become expensive and as such, combinations of the two methods are usually preferred.

PERFORMANCE OF ML MODELS

A key part of machine learning is **assessing the performance** of the developed models.

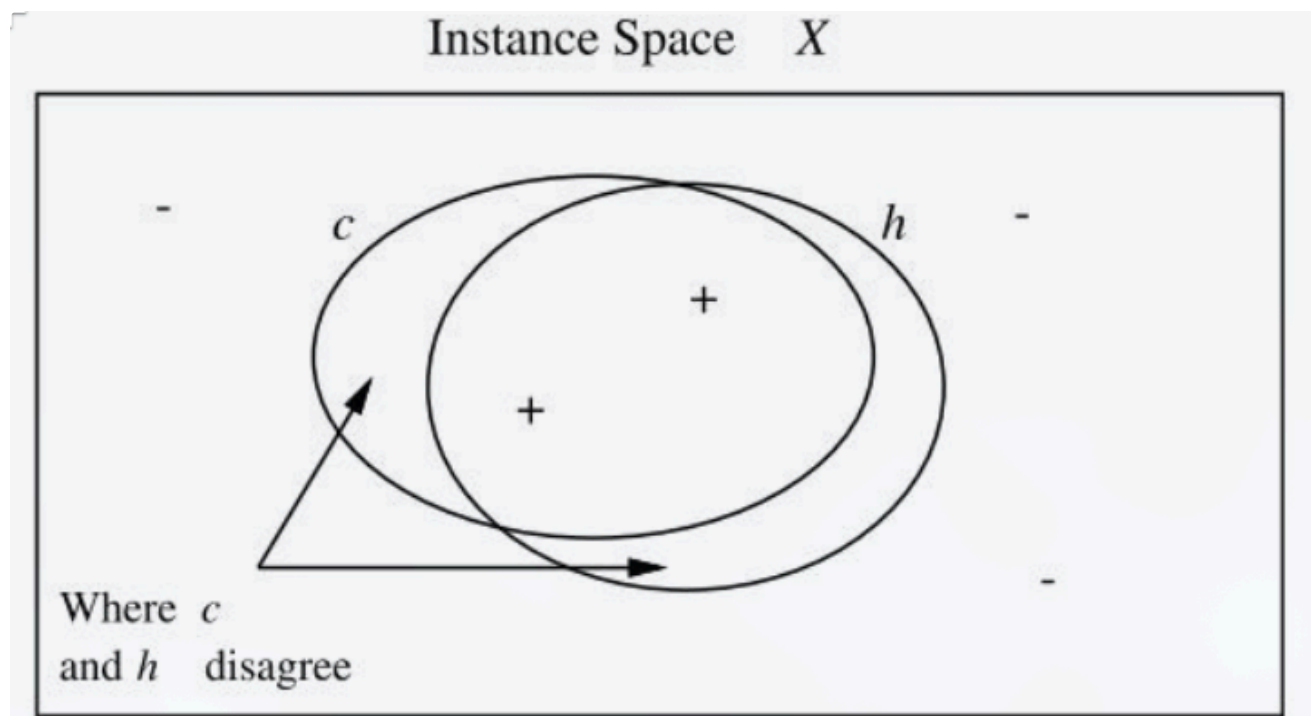
A naive way to *measure the performance* of a model is to use the **classification error** CE and **accuracy** A indicators. These are defined as follows:

$$\text{CE} = \frac{\text{erros}}{\text{total}}$$

$$A = 1 - \text{error} = \frac{\text{correct}}{\text{total}}$$

These are straight forward and easy to understand, but their simplistic nature makes them unsuitable for most applications.

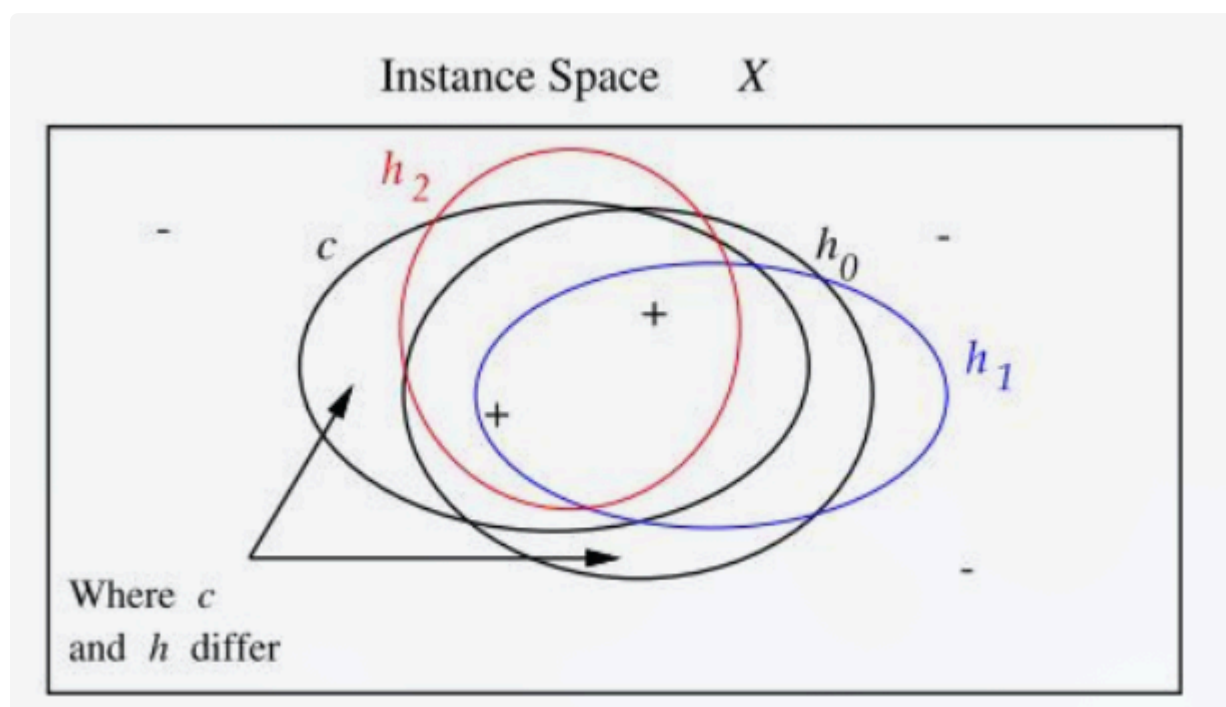
As such, we introduce the definitions of **true error** and **empirical error**.



TRUE ERROR

The **true error** $\text{error}_{\mathcal{D}}(h)$ of hypothesis h with respect to target concept c and distribution $\mathcal{D}$ when observing an input instance $x \in X$ is the **probability that h will misclassify an instance drawn at random** according to $\mathcal{D}$:

$$\text{error}_{\mathcal{D}}(h) := P_{\mathcal{D}}[c(x) \neq h(x)]$$



EMPIRICAL ERROR

The **empirical error** $\text{error}_{\text{Tr}}(h)$ of *hypothesis* h with respect to a set Tr is the **fraction of examples misclassified by h** :

$$\begin{aligned}\text{error}_{\text{Tr}}(h) &:= P_{(x, f(x)) \in \text{Tr}}[f(x) \neq h(x)] \\ &= \frac{|\{(x, f(x)) \in \text{Tr} | f(x) \neq h(x)\}|}{|\text{Tr}|}\end{aligned}$$

OVERFITTING, UNDERFITTING AND GROKING

The definitions given above are useful to define the concept of **overfitting**:

📖 OVERFITTING

Consider an *hypothesis* $h \in \mathcal{H}$ ($\mathcal{H}$ is the hypothesis space, that is, the set of all possible hypotheses) and a training set Tr .

We say that h **overfits** Tr if:

$$\exists h' \in \mathcal{H} \text{ s.t. } \text{error}_{\text{Tr}}(h) < \text{error}_{\text{Tr}}(h') \text{ but } \text{error}_{\mathcal{D}}(h) > \text{error}_{\mathcal{D}}(h')$$

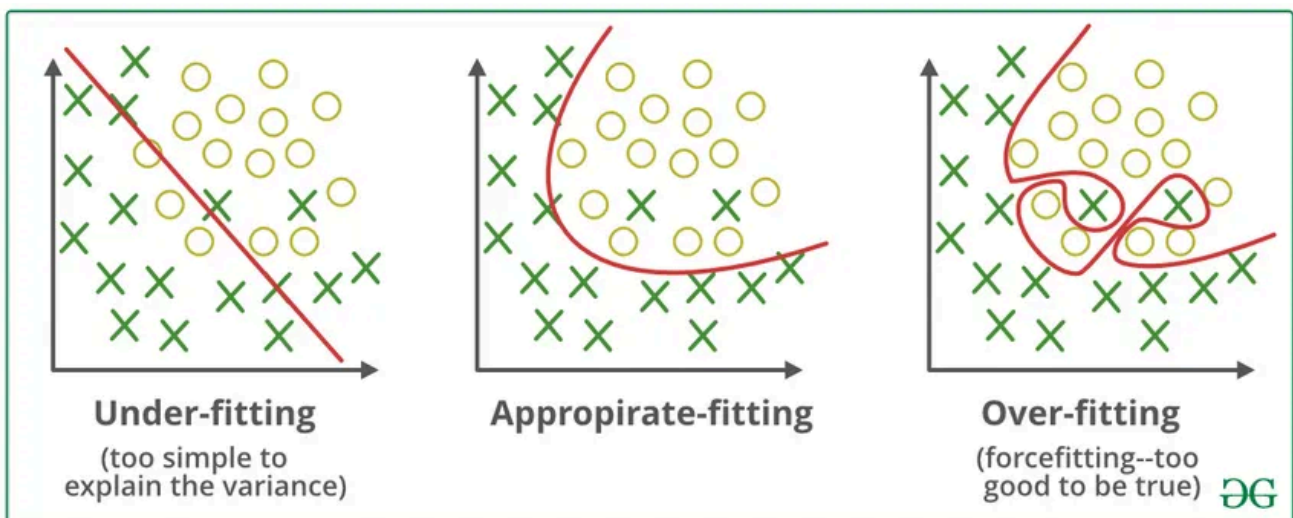
💡 INSIGHT ON THE DEFINITION

Looking at the definition, we see that h *overfits the training set* if there exists **another hypothesis h'** with a **greater empirical error** but a **smaller true error**.

This happens because the model "**learns the training set too well**" and ends up **underperforming in the test set**.

The "opposite" phenomenon is **underfitting**: in this case the **model is too simple to describe the data** (which would require a more complex model).

The following figures are helpful to understand *overfitting* and *underfitting*.



Let's remember that our goal is to **maximise accuracy on test set**.

Assuming that the validation set is representative of the test set, we aim to *maximise accuracy on the validation set*.

In measuring this accuracy, we use the term **generalisation** to refer to the model's ability to *transfer the learned high accuracy from the training set to validation*.

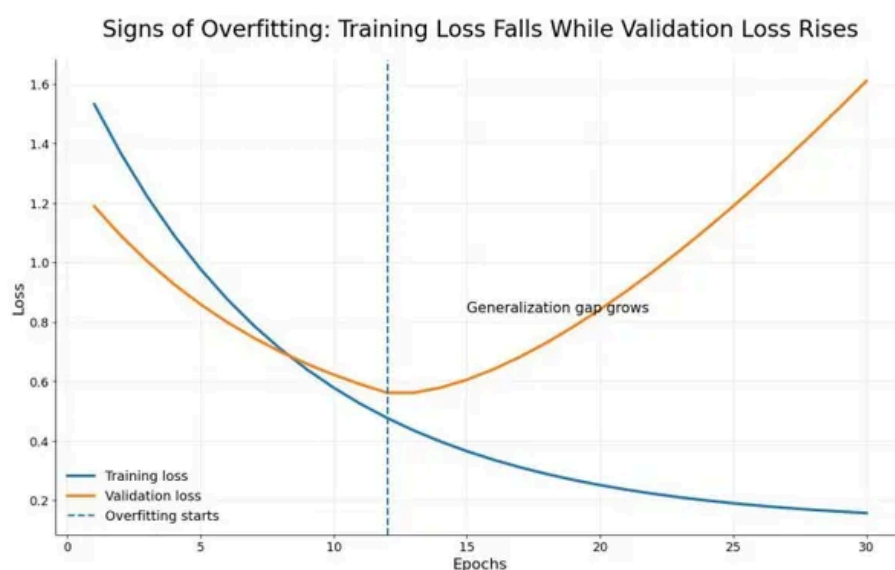
The concept of *generalisation* helps us gain an insight on overfitting and underfitting.

🔗 UNDER-FITTING AND OVER-FITTING CLARIFICATION

The problem of overfitting lies in the fact that the *training set is just a subset of all the data the model will encounter*: describing the training data *too accurately* may result in **poorer results** on the **test set**, where **data isn't perfectly "aligned"** with the training set.

This is what we refer to as **poor generalisation**.

On the other hand, designing a model that's **too simple** may produce **wrong outputs** due to the model's *inability to describe the variance of the data*.



In iterative training processes, after a certain number of iterations we typically notice that the **accuracy on validation no longer increases** (and may begin to decrease) **due to overfitting**.

By *monitoring the progress*, training can be **stopped at the ideal point**.

The *degrees of freedom of the classifier* must not be excessive, but adequate in relation to the complexity of the problem.

It is a good idea to **start with few degrees of freedom** (controllable through hyperparameters) and **gradually increase them** by monitoring accuracy on training and validation sets.

Remember:

All things being equal, the simplest explanation is to be preferred. - William of Occam

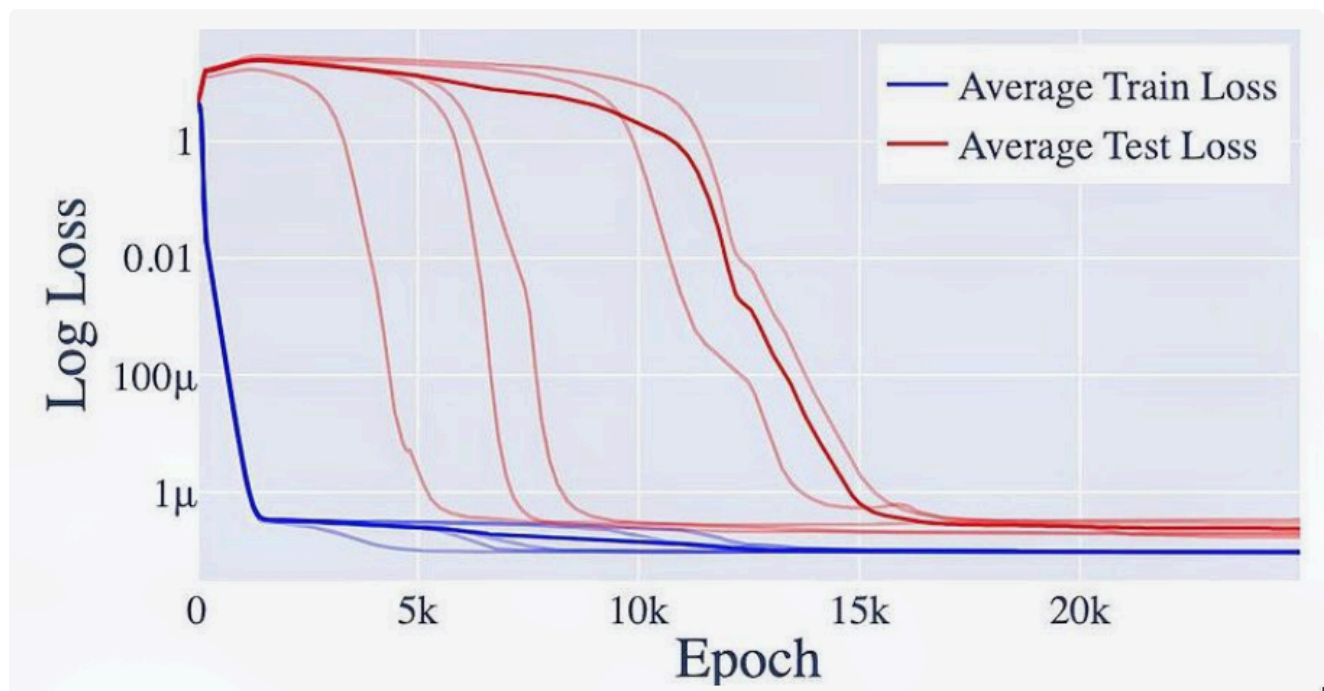
Everything should be made as simple as possible, but not simpler. - Albert Einstein

Another phenomenon related to the training of machine learning models is known as **grokking**.

📖 GROKKING

Intriguing learning phenomenon where **test set loss (error) decreases sharply only after a model's training set loss has converged**.

Grokking is a phenomenon where a neural network *first memorises the training data* and shows *poor generalisation*, but **after prolonged training far beyond the point of overfitting**, it suddenly and sharply **improves its generalisation** capabilities. This leads to a *long plateau of low test accuracy* followed by an *abrupt jump*, even though *training accuracy was already high*.



📝 NOTE

Late generalisation **challenges traditional stopping criteria**, which rely on monitoring training and test set losses, but it **remains difficult to predict when** late generalisation might occur.

OPEN SET PERFORMANCE INDICATORS

In many real cases, patterns to be classified may not just belong to **one of the known classes**; they might **not belong to any of them**.

These are called **open set problems**.

A simple example would be classifying fruit into {apple, pear, other}.

As suggested by the example, a possible approach for this kind of problems is **adding another fictitious class** that represents "*the rest of the world*" (in the example, this is called *other*) is added to the classes and the "negative examples" (data belonging to class "other") are added to the training set.

Another possible solution to open set problems is achieved by **allowing** the system to **not assign a class** to a pattern.

To this end, a **threshold** is defined and the *pattern is assigned* to the most probable class *only when the probability is higher than the threshold*.

In both cases, to assess the model's ability to correctly **distinguish between known and unseen classes**, we can consider the following diagram:

		Actual	
		Positive	Negative
Predicted	Positive	True Positive	False Positive
	Negative	False Negative	True Negative

A **true positive** is an outcome where the model *correctly predicts the positive class*. Similarly, a **true negative** is an outcome where the model *correctly predicts the negative class*.

A **false positive** is an outcome where the model *incorrectly predicts the positive class*, and a **false negative** is an outcome where the model *incorrectly predicts the negative class*.

⚠ ATTENTION

1. With "*positive class*" we are referring to *all the known classes*; thus, the *negative class* corresponds to the *unseen classes*.
2. These indicators **are not specific to open set problems**, they can also be used in other contexts.

Let:

- TP be the *number of true positives*.
- FP be the *number of false positives*.
- TF be the *number of true negatives*.
- FN be the *number of false negatives*.
- N_p be the *number of patterns belonging to the positive class*.
- N_n be the *number of patterns belonging to the negative class*.
- P_p be the *number of patterns labelled as positive*.
- P_n be the *number of patterns labelled as negative*.

Thanks to the diagram above we can define the following indicators:

- **TRUE POSITIVE RATE** - Also known as **recall**, *sensitivity* or *power*, it's the *probability of detection* of the positive class: $TPR = \frac{TP}{N_p}$.
- **FALSE NEGATIVE RATE** - Also known as *miss rate*: $FNR = \frac{FN}{N_p}$.
- **FALSE POSITIVE RATE** - Also known as *fall out*, it's the *probability of false alarm*: $FPR = \frac{FP}{N_n}$.
- **TRUE NEGATIVE RATE** - Also known as *specificity*: $TNR = \frac{TN}{N_n}$.
- **PRECISION** - Also known as *positive predictive value*: $Pre = \frac{TP}{P_p}$.
- **FALSE DISCOVERY RATE** - $FDR = \frac{FP}{P_p}$.
- **FALSE OMISSION RATE** - $FOR = \frac{FN}{P_n}$.
- **NEGATIVE PREDICTIVE VALUE** - $NPV = \frac{TN}{P_n}$.

Once these are defined, we can also calculate:

- **POSITIVE LIKELIHOOD RATIO** - $LR_+ = \frac{TPR}{FPR}$.
- **NEGATIVE LIKELIHOOD RATIO** - $LR_- = \frac{FNR}{TNR}$.
- **DIAGNOSTIC ODDS RATIO** - $DOR = \frac{LR_+}{LR_-}$.
- **F1 SCORE** - $F_1 = 2 \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$.

💡 OBSERVATION

False positive rate and false negative rate are a **function of the threshold t** .
High *restrictive thresholds* reduce false positives but increase the false

negatives, while *more tolerant thresholds* reduce false negatives but increase the false positives.

TRAINING, VALIDATION AND TESTING

We mentioned *training*, *validation* and *test sets* more than a couple times, so let's properly define them.

TRAINING, VALIDATION AND TEST SETS

- **TRAINING SET** (*train*) - The set of patterns used to train the system, *finding the optimal value for the parameters*.
- **VALIDATION SET** (*valid*) - The set of patterns on which to *set the hyperparameters*.
- **TEST SET** (*test*) - The set of patterns on which to *evaluate the final system performance*.

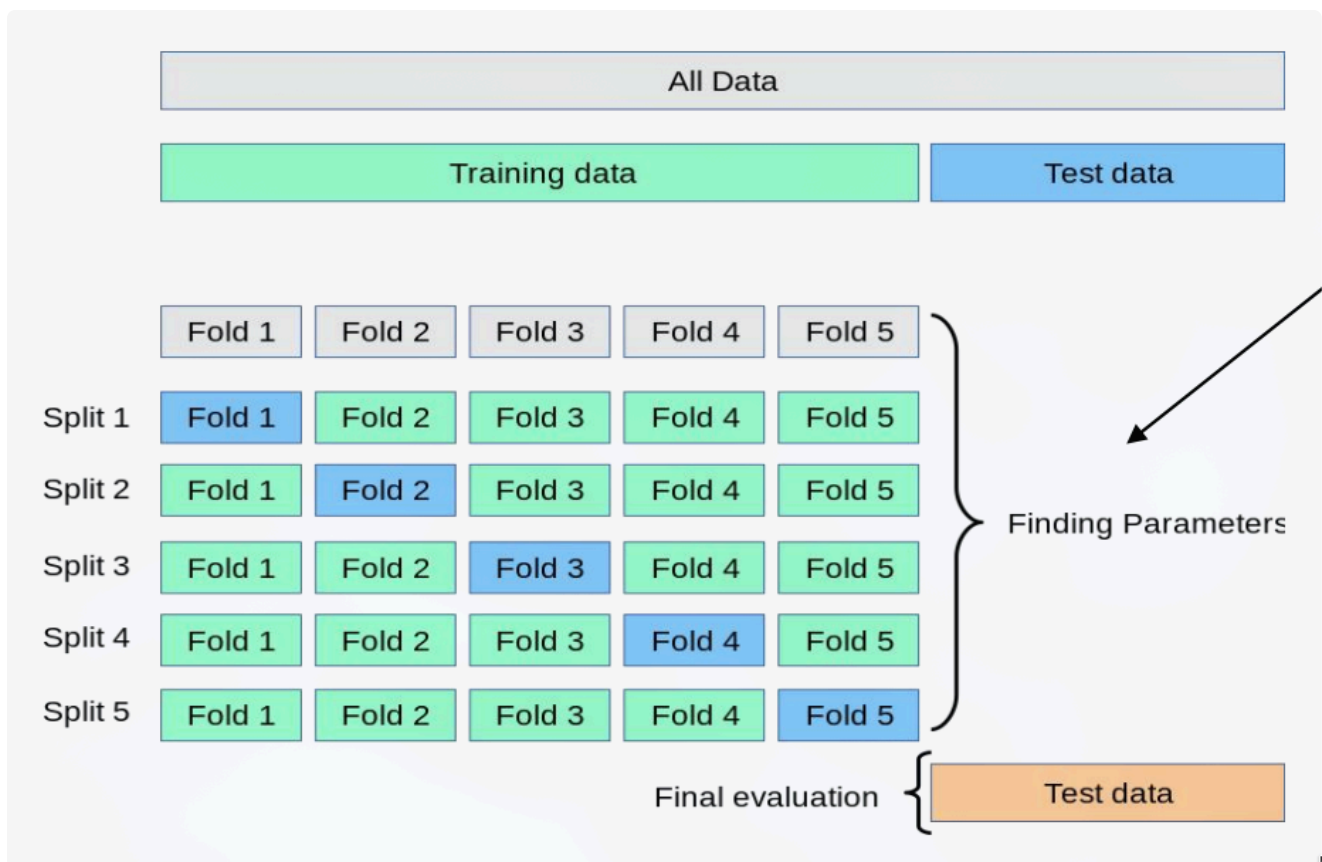
In machine learning benchmarks, the split of the patterns into Train, Valid and Test is often predefined to make the results comparable.

Warning

Never use the **test set** to **choose parameters**: these are optimised solely on the basis of the *training set*.

The **test set** must **only be used** for the final **validation of performance**.

One way to obtain the three sets from the entire available data set is seen in the figure below:



The data set is **first split in two subsets**: the *training set* (larger of the two subsets) and the *test set*.

Then, the *training data* is **split into n "folds"** (5 in the example).

To optimise the model's parameters, we repeat n **iterations**: each iteration utilises $n - 1$ folds as *training data* and the *last fold* as *validation set*.

At the end, we *average the validation scores* obtained and *pick the hyperparameters that resulted in the best average performance*.

Finally, we perform the *model evaluation* on the *test set*.

⚠ NOTE

This is just *one of the possible* approaches: there are many, more complicated ways to split the original data into the three sets.

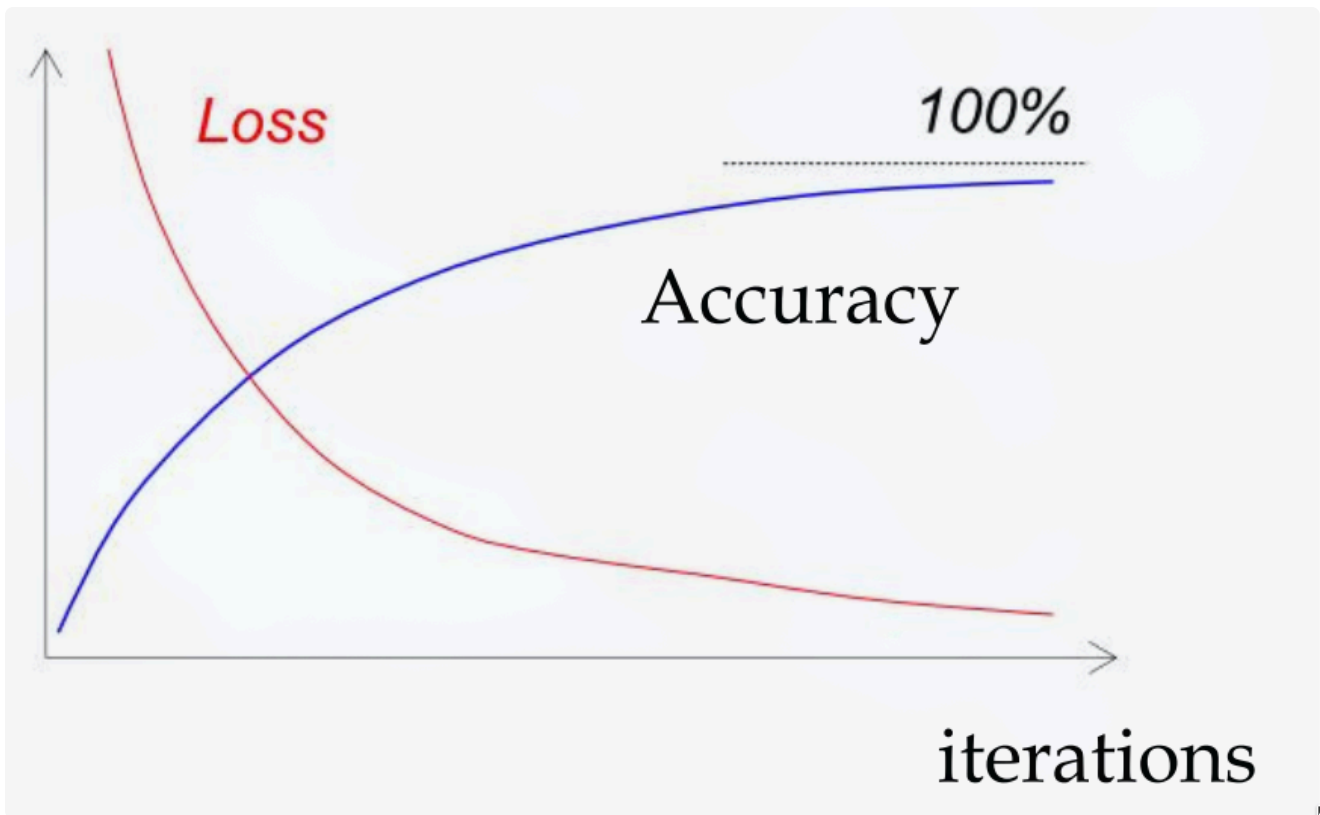
Interested readers can look up *bayesian optimisation for hyperparameter tuning*.

CONVERGENCE

The first aim to be pursued during the training is the **convergence on the training set**.

Let's consider a classifier whose training involves an iterative process.

Convergence **occurs** when **loss** (output of the loss function) **has a decreasing trend** and thus accuracy has an increasing trend.



If *loss does not decrease* (or fluctuates significantly) the system **does not converge**: the *optimisation method is not effective*, the *hyperparameters are out of range*, the *learning rate is inadequate*, there are *implementation errors*, etc.

If the **loss decreases** but the **accuracy does not increase**, the most probable cause is a **wrong choice of loss-function**.

If the **accuracy does not approach 100%** on the training set, the **degrees of freedom of the classifier are not sufficient** to manage the complexity of the problem.